Creational Design Patterns

1. How do you implement a thread-safe Singleton?

The safest and easiest way is to use a Java Enum, as the JVM guarantees thread safety automatically and handles all initialization details. Alternatively, you can use a static field initialized immediately (Eager Initialization), or use Double-Checked Locking with the volatile keyword, which requires manual synchronization in the getInstance() method.

2. How do you prevent Singleton breaking via reflection and serialization?

To stop reflection attacks, check inside the private constructor and throw an exception if the instance is already created. To prevent serialization from creating a new instance, you must implement the readResolve() method. This method forces the JVM to return the existing instance instead of the newly deserialized object.

3. When should you avoid Singleton?

Avoid Singleton when the object needs to hold mutable data (state that changes), as this acts like a global variable and makes testing very difficult. It also creates tight coupling, making your system rigid and impossible to test a class in isolation without the Singleton affecting the result.

4. Factory vs Abstract Factory : When to choose what?

Choose the Factory Method when you need to create a *single* product type and want subclasses to decide the exact implementation to instantiate (e.g., creating a Notification object, letting the subclass choose Email or SMS). Choose the Abstract Factory when you need to create *families* of related products (e.g., creating a matching set of dark-themed UI components) and consistency across the family is critical.

5. Builder vs Factory : which is better for complex object creation?

The Builder pattern is better for complex object creation, especially when the object has many optional parameters. The Factory pattern is better when the object is created in a single step, and the complexity lies in deciding *which* class to create based on input. Builder focuses on *how* to construct step-by-step; Factory focuses on *what* to create in one go.

6. Prototype pattern vs just cloning vs creating new instance?

The Prototype pattern is a general guideline to create new objects by cloning an existing one (the prototype) when object creation is expensive. You choose *cloning* over creating a new instance when the cost of initializing a fresh object (like fetching data from a network) is significantly higher than copying the existing object's state. You must ensure you implement deep cloning if the object contains mutable references that should not be shared.

STRUCTURAL PATTERNS

. Adapter vs Decorator vs Facade vs Proxy : Detailed Comparison

Pattern	Goal (Purpose)	Interface Behavior	Core Mechanism
Adapter	Compatibility	Changes a mismatched interface to one the client expects.	Wraps one class to *convert* its methods.
Decorator	Enhancement	Keeps the interface the same while adding new responsibilities.	Wraps the object to *add* features dynamically.
Facade	Simplification	Provides a single, simple interface to a complex subsystem.	Wraps *many* classes to *hide* complexity.
Proxy	Control/Protection	Keeps the original interface but controls access to the real object.	Wraps the real object to *manage* requests.

8. When do you use the Bridge pattern?

You should use the Bridge pattern when you need to separate the Abstraction (what the system does) from the Implementation (how it does it) so they can change independently. It is best used to avoid a "class explosion" when you have multiple dimensions of variation (like needing classes for *Red-Circle*, *Blue-Circle*, *Red-Square*, etc.). Instead, you connect a "Shape" to a "Color" using a bridge, reducing the number of classes needed.

9. Composite pattern : How to model hierarchical structures?

The Composite pattern is used to represent tree-shaped or nested structures where you want to treat individual objects (like a "File") and groups of objects (like a "Folder") in exactly the same way. The pattern achieves this by making both individual items and groups implement a

common interface. This allows you to write one simple method (like calculateSize()) that works uniformly on every node in the hierarchy.

10. How does Decorator add behavior dynamically?

The Decorator pattern adds behavior dynamically by composition. When you create a decorator, you pass the original object to the decorator's constructor. The decorator maintains the original object's interface but adds its own custom logic before or after calling the original object's method. This allows you to stack multiple features (decorators) onto an object at runtime.

11. When is Facade an anti-pattern?

A Facade becomes an anti-pattern (often called a "God Object") when it starts taking on too much responsibility and contains business logic itself, rather than just delegating. A Facade's only job should be to simplify access to other classes. If it grows too large and complex, it becomes a single bottleneck that is difficult to maintain and test.

12. Real-world use of Proxy (Spring AOP, Hibernate Lazy Loading)

Proxies are widely used in frameworks to provide control without altering the original class code. Spring uses proxies for Aspect-Oriented Programming (AOP); for example, a proxy intercepts a method call to apply the @Transactional logic before the actual method runs. Hibernate uses proxies for Lazy Loading, where a placeholder object is created, and the data is only fetched from the database when you first access that object's fields.

BEHAVIORAL PATTERNS

13. Strategy vs Template Method : Differences and Use-Cases

Pattern	Mechanism	Flexibility	When to Use
Strategy	Composition (has-a relationship)	High. You swap the entire algorithm object at runtime.	When the *method* must change instantly (e.g., choosing CreditCardPayment vs PayPalPayment).

Pattern	Mechanism	Flexibility	When to Use
Template Method	Inheritance (is-a relationship)	Low. The overall sequence/structure of the algorithm is fixed.	When the *process structure* must be enforced (e.g., ensuring all data processes follow "connect -> read -> close").

In simple terms: Strategy lets you change *what* you do by plugging in a different helper object. Template Method locks down *the order* of operations and only lets you change the specific details of certain steps.

14. Observer vs Pub/Sub : What's the Practical Difference?

The core goal of both is the same (notification), but the architectural pattern is different. Observer is a direct pattern: the subject (publisher) knows and maintains a list of the specific observer objects that need to be notified. Pub/Sub (Publisher/Subscriber) is indirect and usually requires a dedicated Message Broker (e.g., Redis, Kafka). The publisher publishes an event to a channel, and the broker ensures all subscribed listeners receive it. This decouples the publisher and subscriber completely.

15. Challenges with Observer in Multithreading?

The Observer pattern is not thread-safe by default. If one thread is notifying the list of observers while another thread is simultaneously trying to add or remove an observer, it can lead to race conditions and data corruption. This often results in a ConcurrentModificationException. To fix this, all methods related to subscription and notification must be properly synchronized.

16. How to Design Undo/Redo using Command Pattern?

The Command pattern makes undo/redo simple because it wraps every action into a specific Command object. These command objects are stored sequentially in a history stack (or queue). To implement undo, you simply retrieve the last command from the stack and call its corresponding unExecute() method, which reverses the operation.

17. Chain of Responsibility : Where is it Useful? What are Pitfalls?

This pattern is useful when a request needs to be processed, but you don't know exactly which object should handle it. Common uses include authentication filters (passing a request through security checks) or logging (passing a message until a handler processes that severity level). The main pitfall is that there is no guarantee the request will ever be handled if the chain is set up incorrectly, causing the request to simply disappear.

SOLID PRINCIPLES

18. Give a real example where SRP was violated and you refactored.

I once had a class called ProductService that handled fetching product data, generating sales reports, and sending inventory alerts. This violated **SRP** because it had three reasons to change. I refactored it by splitting the work into three classes: ProductRepository (data access), SalesReporter (report calculation), and InventoryAlerter (communication). This ensured changes to the email format didn't require touching the secure product data logic.

19. How OCP helps in real enterprise code?

The **Open/Closed Principle (OCP)** ensures stability when adding features. For example, if an enterprise system needs to add a new ExpressShipping method, OCP requires creating a *new* ExpressShipping class that extends the base logic. You avoid modifying the existing, working StandardShipping class, which prevents introducing new bugs into old, stable features.

20. What are signs of LSP violation?

The biggest sign is when the client code that uses an object has to check its actual type using **instanceof** to decide what to do. Another clear sign is when a subclass throws a new, unexpected exception that the client code didn't know how to handle. This breaks the fundamental expectation that the subtype can be seamlessly swapped with the parent type without causing errors.

21. How DIP improves testability in a large application?

The **Dependency Inversion Principle (DIP)** requires high-level code to rely on **interfaces** instead of concrete classes. This allows you to easily substitute complex dependencies (like a real database) with fake objects (a **mock**) during unit tests. You can test your business logic fast and reliably without needing to set up the actual external environment.

22. Practical examples of ISP in microservices.

The **Interface Segregation Principle (ISP)** suggests creating small, client-specific contracts. In a microservice architecture, this means the **API Gateway** should expose tailored endpoints for specific clients. For example, the Mobile app might see a small /user-profile endpoint, preventing it from having to depend on data only related to the much larger /admin-user-details endpoint.